

AutoCal

System automatycznego zarządzania planem zajęć w Google
Calendar

Autor: Paweł Hase

Wydział: WEEIA
Uczelnia: Politechnika Łódzka

Spis treści

1	Wprowadzenie	4
1.1	Cel projektu	4
1.2	Technologie	4
2	Architektura systemu	4
2.1	Komponenty mikroservisów	4
2.2	Wzorzec Worker Pattern	4
2.3	Komunikacja między serwisami	5
3	Komponenty systemu	5
3.1	CLI Container (cli)	5
3.2	Processor Container (processor)	5
3.3	Data-Prepare Container (data-prepare)	5
3.4	Google Calendar API Container (gcal-api)	5
4	API i CLI	6
4.1	Endpointy API	6
4.1.1	/config (POST)	6
4.1.2	/export (POST)	6
4.2	Komendy CLI	6
4.2.1	register	6
4.2.2	config	6
4.2.3	export	6
5	Baza danych i kolejki	6
5.1	MongoDB - Kolekcja configurations	6
5.2	Redis Queues	7
6	Diagramy systemu	7
6.1	Diagram STD - Proces selekcji zajęć (Data-Prepare Worker)	7
6.2	Diagram DFD - Przepływ danych między mikroservisami	8
7	Deployment i skalowalność	10
7.1	Docker Compose	10
7.2	Skalowanie Worker Pattern	10
7.3	Sieci i wolumeny	10
8	Podsumowanie	11
8.1	Osiągnięte cele	11
8.2	Kluczowe wzorce architektoniczne	11
8.3	Możliwości rozwoju	11
8.4	Repozytorium	11

Spis rysunków

1	Diagram STD procesu selekcji zajęć	8
2	Diagram DFD przepływu danych między mikroserwisami	9

Spis tabel

1	Schemat kolekcji <code>configurations</code> w MongoDB	7
---	--	---

1 Wprowadzenie

AutoCal to system automatycznego zarządzania planem zajęć, który umożliwia integrację planu zajęć studenta z harmonogramem wydziałowym i eksport wydarzeń do Google Calendar. System wykorzystuje architekturę mikroservisów opartą na kontenerach Docker.

1.1 Cel projektu

Głównym celem projektu jest automatyzacja procesu tworzenia i synchronizacji planu zajęć z kalendarzem Google, eliminując konieczność ręcznego wprowadzania zajęć przez studentów.

1.2 Technologie

System został zbudowany z wykorzystaniem następujących technologii:

- **Konteneryzacja:** Docker, Docker Compose
- **Backend:** Python, FastAPI
- **Baza danych:** MongoDB, Redis
- **API zewnętrzne:** Google Calendar API
- **Wzorce architektoniczne:** Microservices, Worker Pattern

2 Architektura systemu

2.1 Komponenty mikroservisów

System składa się z sześciu głównych komponentów:

1. **CLI Container** – Interfejs wiersza poleceń
2. **Processor Container** – Główny serwer API
3. **Data-Prepare Container** – Worker przetwarzający dane
4. **Google Calendar API Container** – Worker obsługujący Google Calendar
5. **MongoDB** – Baza danych dokumentowa
6. **Redis** – System kolejek i cache

2.2 Wzorzec Worker Pattern

System implementuje wzorzec Worker Pattern, który zapewnia następujące korzyści:

- **Asynchroniczne przetwarzanie:** Długotrwałe operacje wykonywane w tle
- **Skalowalność:** Możliwość uruchomienia wielu instancji worker'ów
- **Niezawodność:** Izolacja błędów między komponentami
- **Wydaźność:** Równoległy processing zadań

2.3 Komunikacja między serwisami

Komunikacja w systemie odbywa się przez:

- **HTTP REST API:** Dla interakcji CLI z Processor
- **Redis Queues:** Dla asynchronicznej komunikacji między worker'ami
- **Shared Volumes:** Dla współdzielenia plików konfiguracyjnych

3 Komponenty systemu

3.1 CLI Container (cli)

Funkcjonalności:

- Rejestracja Google OAuth
- Upload plików JSON (schedule, timetable)
- Eksport planu zajęć

Kluczowe komponenty: click, requests, google-auth-oauthlib

3.2 Processor Container (processor)

Funkcjonalności:

- Walidacja JSON (schedule/timetable względem schematów)
- Zarządzanie konfiguracją MongoDB
- Koordynacja zadań między worker'ami
- Obsługa endpointów REST API

3.3 Data-Prepare Container (data-prepare)

Algorytm przetwarzania:

1. Pobranie dni roboczych (poniedziałek-piątek)
2. Mapowanie dat na tygodnie harmonogramu
3. Filtrowanie zajęć według występowania
4. Konwersja do formatu Google Calendar Event

3.4 Google Calendar API Container (gcal-api)

Funkcjonalności:

- Autoryzacja OAuth Google
- Tworzenie kalendarza "AutoCal"
- Batch dodawanie wydarzeń
- Obsługa błędów API

4 API i CLI

4.1 Endpointy API

4.1.1 /config (POST)

- **Input:** token.json, schedule.json, timetable.json
- **Output:** config_id (UUID4)
- **Proces:** Walidacja → MongoDB → config_id

4.1.2 /export (POST)

- **Input:** config_id, offset (-1/0/1 dla poprzedni/aktualny/następny tydzień)
- **Output:** status eksportu + liczba wydarzeń
- **Proces:** MongoDB → data-prepare → Google Calendar → status

4.2 Komendy CLI

4.2.1 register

```
1 python cli.py register
```

Listing 1: Autoryzacja Google OAuth

Autoryzacja Google OAuth (port 2255) → token.json

4.2.2 config

```
1 python cli.py config
```

Listing 2: Konfiguracja systemu

Upload plików → POST /config → config_id.pkl

4.2.3 export

```
1 python cli.py export -o 1 # nastepny tydzien
```

Listing 3: Eksport planu zajęć

POST /export → status eksportu

5 Baza danych i kolejki

5.1 MongoDB - Kolekcja configurations

Struktura dokumentu przedstawiona w tabeli [1](#).

Tabela 1: Schemat kolekcji `configurations` w MongoDB

Pole	Typ	Opis
<code>_id</code>	UUID4	Unikalny identyfikator
<code>tk</code>	JSON	Token Google OAuth
<code>schedule</code>	Object	Harmonogram wydziałowy
<code>timetable</code>	Object	Plan zajęć studenta
<code>created_at</code>	DateTime	Data utworzenia (UTC)

5.2 Redis Queues

System wykorzystuje następujące kolejki Redis:

- `data_queue` / `RES:data_queue`: Zadania/odpowiedzi `data-prepare`
- `google_api_queue` / `RES:google_api_queue`: Zadania/odpowiedzi Google Calendar

Wzorzec Request-Response:

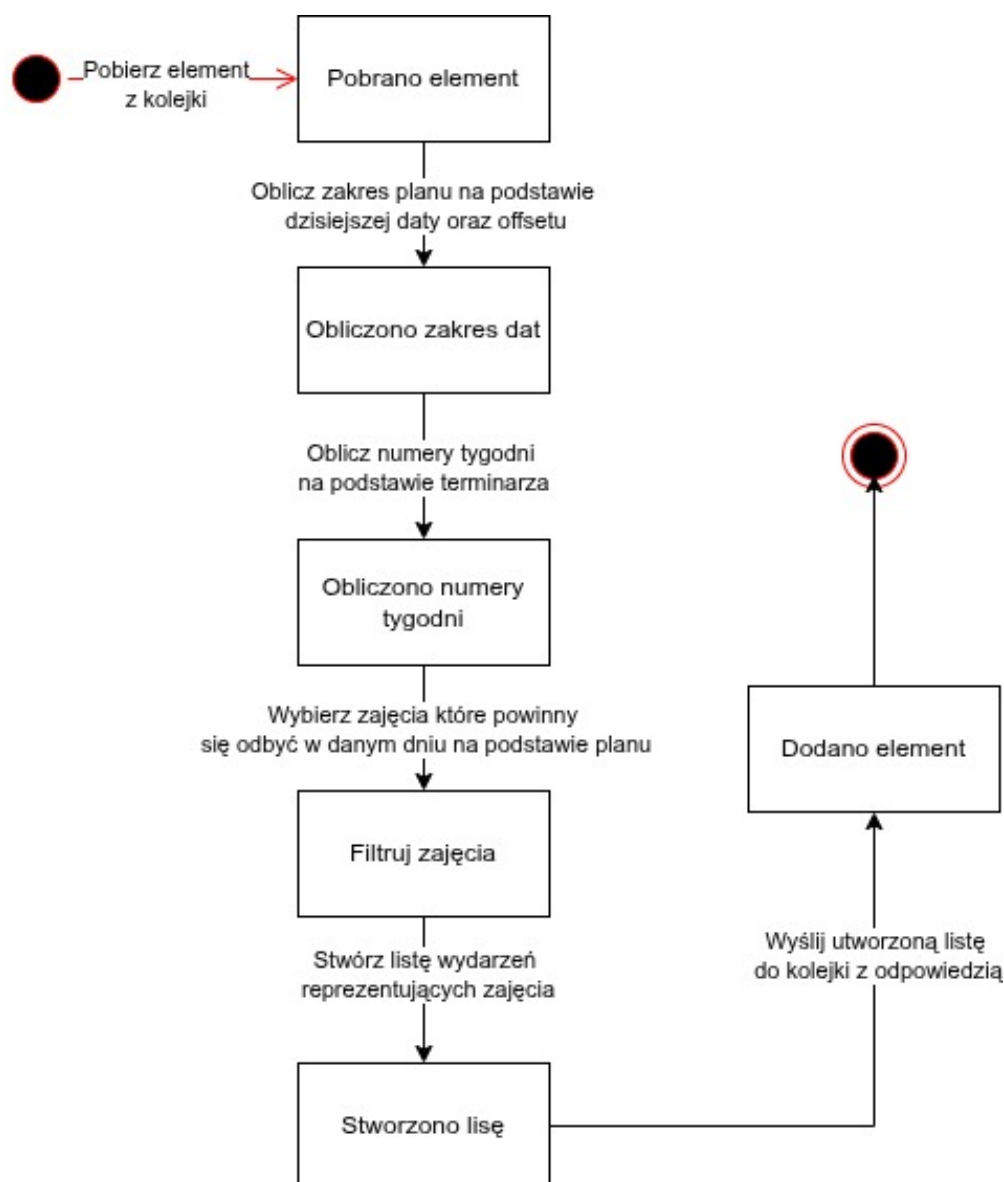
```
1 redis_client.rpush(QueueName, json.dumps(task_data))
2 response = redis_client.blpop(f"RES:{QueueName}", timeout=30)
```

Listing 4: Implementacja komunikacji przez Redis

6 Diagramy systemu

6.1 Diagram STD - Proces selekcji zajęć (Data-Prepare Worker)

Na rysunku 1 przedstawiono diagram przejść stanów dla procesu selekcji zajęć w kontenerze `data-prepare worker`.

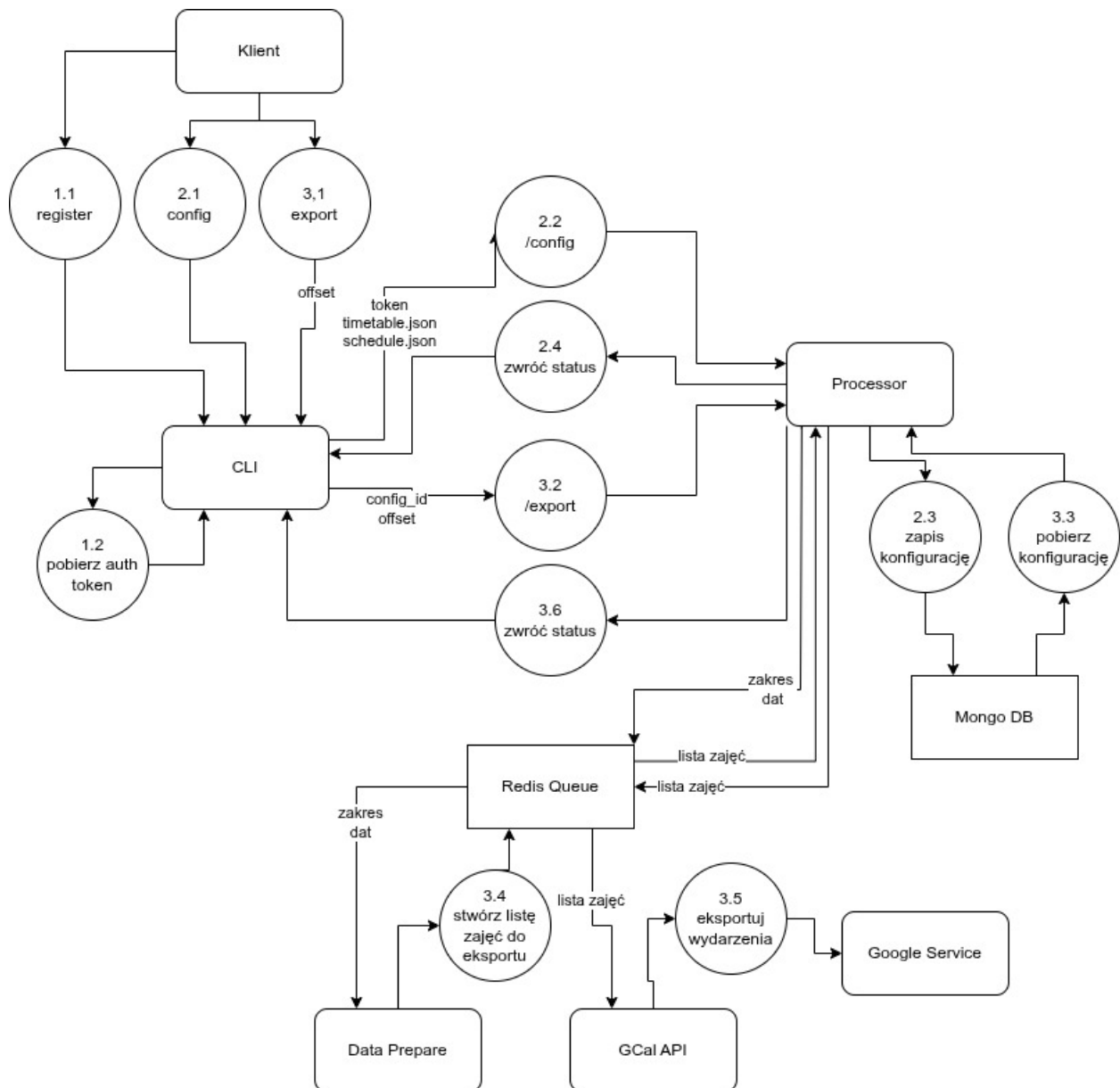


Rysunek 1: Diagram STD procesu selekcji zajęć

Diagram przedstawia proces selekcji zajęć odbywających się w określonym okresie czasu w kontenerze data-prepare worker po wywołaniu komendy `export`. Proces obejmuje walidację danych, sprawdzanie formatów dat, kontrolę mapowania tygodni oraz obsługę błędów konwersji.

6.2 Diagram DFD - Przepływ danych między mikroserwisami

Rysunek 2 przedstawia przepływ danych między wszystkimi komponentami systemu.



Rysunek 2: Diagram DFD przepływu danych między mikroservisami

Przepływ danych w systemie:

- **User** → **CLI** → **Processor**: Komendy i HTTP API calls
- **Processor** → **Workers (Redis)**: Zadania i odpowiedzi przez kolejki
- **External Services**: MongoDB (storage), Google Calendar API (events)

7 Deployment i skalowalność

7.1 Docker Compose

Konfiguracja Docker Compose przedstawiona w listingu 5.

```
1 services:
2   cli:
3     ## Port 2255 (OAuth)
4   processor:
5     ## Port 8000 (API)
6   data-prepare:
7     ## Worker (scalable)
8   google:
9     ## Worker (scalable)
10  mongo:
11    ## Port 27017
12  redis:
13    ## Port 6379
```

Listing 5: Konfiguracja Docker Compose

7.2 Skalowanie Worker Pattern

```
1 # Skalowanie worker'ów
2 docker-compose up --scale data-prepare=3 --scale google=2
```

Listing 6: Skalowanie worker'ów

Zalety:

- **Horizontal scaling:** Wiele instancji worker'ów
- **Load distribution:** Automatyczne rozdzielanie zadań
- **Fault tolerance:** Izolacja błędów

7.3 Sieci i wolumeny

- **Network:** autocal-network (bridge) – izolacja i DNS
- **Shared volumes:** ./shared (biblioteka), ./cli (dane)
- **Persistent volumes:** mongo_data, redis_data

8 Podsumowanie

8.1 Osiągnięte cele

System AutoCal z powodzeniem implementuje:

- ✓ **Automatyzacja:** Eksport planu zajęć do Google Calendar
- ✓ **Mikroserwisy:** Architektura Docker containers
- ✓ **Skalowalność:** Worker pattern z Redis queues
- ✓ **CLI Interface:** Intuicyjne komendy (register, config, export)
- ✓ **Error handling:** Walidacja JSON i obsługa błędów API

8.2 Kluczowe wzorce architektoniczne

- **Microservices:** Podział na niezależne kontenery
- **Worker Pattern:** Asynchroniczne przetwarzanie przez Redis
- **Request-Response:** Kolejki z timeout'ami
- **Container Orchestration:** Docker Compose management

8.3 Możliwości rozwoju

Potencjalne kierunki rozwoju systemu:

- Web interface oprócz CLI
- Notyfikacje o zmianach w planie
- Monitoring i logging system
- API rate limiting i caching

8.4 Repozytorium

Kod źródłowy projektu dostępny pod adresem:

<https://github.com/hallu-boss/AutoCal>